CYBERSECURITY INTERNSHIP | WEEK 5 TASK

Vulnerability Assessment and Penetration Testing (VAPT)

Practical Report

Network scanning with Nmap, web-server scanning with Nikto and HTTP traffic analysis with Burp Suite

Student name	Tanmay Sarkar
Institution	Tezpur University, B.Tech Computer Science Engineering
Task	Week 5: VAPT Practical
Targets tested	testphp.vulnweb.com (Nmap) Locally hosted DVWA at http://127.0.0.1/dvwa/ (Nikto, Burp Suite)
Tools used	Nmap 7.98, Nikto 2.6.0, Burp Suite Community v2026.7.2, curl, MariaDB client
Test platform	Kali Linux
Testing date	1 October 2026
Submission date	

1. Abstract and Introduction

Vulnerability Assessment and Penetration Testing (VAPT) is a structured process for discovering, classifying and, where authorized, safely verifying security weaknesses in systems, networks and web applications. A vulnerability assessment finds and rates weaknesses; penetration testing checks whether they can actually be abused. Together they show an organization where it is exposed before an attacker finds out.

This practical applies the basic VAPT workflow with three standard tools: **Nmap** for port, service and operating-system discovery, **Nikto** for common web-server misconfigurations, and **Burp Suite** for observing HTTP requests and responses. Every tool result is interpreted, checked against manual evidence where possible, rated for risk and paired with a recommendation. Scanner output is treated as a lead to be validated, not as proof of exploitability.

1.1 Summary of results

Tool	Target	Outcome
Nmap	testphp.vulnweb.com (44.228.249.3)	All 1000 scanned TCP ports reported as filtered (no response). No services or versions identified; OS guesses unreliable.
Nikto	DVWA on 127.0.0.1 (Apache/2.4.68)	20 items reported in 8040 requests: missing security headers, directory indexing, exposed .git and ignore files.
Burp Suite	DVWA on 127.0.0.1 (proxy 127.0.0.1:8080)	Login POST and authenticated page request captured; unencrypted HTTP, cookie attributes and status codes reviewed.
Manual checks	DVWA on 127.0.0.1	curl header and cookie review, Apache error log, MariaDB and application configuration review.

In total **nine evidence-based findings** are documented in Section 9: three Medium, four Low and two Informational. DVWA (Damn Vulnerable Web Application) is a deliberately vulnerable training application, so the ratings describe each issue as if the same configuration were exposed in production.

2. Scope, Methodology and Environment

2.1 Scope and authorization statement

Testing was restricted to systems that are public test targets or owned by the tester:

- **Nmap** was run against `testphp.vulnweb.com`, the Acunetix public test website designated as the authorized target for this practical.
- **Nikto and Burp Suite** were run against a DVWA instance installed on the tester's own Kali Linux machine (`http://127.0.0.1/dvwa/`). This is a student-owned environment and is within the scope allowed for the practical.

No unrelated domains or IP addresses were scanned. No denial-of-service testing, brute-forcing, exploitation, data modification or malware upload was performed.

2.2 Test environment

Item	Details
Operating system	Kali Linux (user: tanmay), working directory ~/Week-5-VAPT with sub-folders Nmap, Nikto, Burp, Screenshots, Report, PPT
Network scanner	Nmap 7.98

Item	Details
Web-server scanner	Nikto v2.6.0
Proxy / HTTP analysis	Burp Suite Community Edition v2026.7.2; proxy listener 127.0.0.1:8080
Browser	Chromium-based browser v150 (from the User-Agent header captured in Burp)
Local web stack	Apache/2.4.68 (Debian), MariaDB 11.8.8, DVWA (security cookie value: impossible)
Date and time zone	1 October 2026, IST (UTC+5:30)

2.3 Methodology

Stage	What was done
1. Planning	Confirmed targets, tools and limits (Section 2.1).
2. Discovery	Resolved the target IP, checked reachable ports and services with Nmap; started and verified the local Apache and MariaDB services for DVWA.
3. Scanning	Nikto web-server scan of /dvwa/; Burp Suite capture of normal browser traffic.
4. Validation	Re-ran Nikto to check repeatability; confirmed headers and cookies manually with curl; reviewed logs and configuration.
5. Risk analysis	Rated each finding by impact, likelihood, evidence and affected component.
6. Reporting	Documented evidence, impact and remediation for each finding (Section 9).

3. Nmap: Network Scanning

Nmap discovers hosts, ports, services, versions and operating systems. Three scan types were used: `-sS` (TCP SYN scan, port states), `-sV` (service and version detection on open ports) and `-A` (aggressive: OS detection, version detection, default scripts and traceroute). Output was also saved to files using `-oN`.

3.1 Commands executed

Command	Duration	Result
`sudo nmap -sS testphp.vulnweb.com`	56.04 s	1000 filtered TCP ports (no-response)
`sudo nmap -sV testphp.vulnweb.com`	8.46 s	No open ports, so nothing to fingerprint
`sudo nmap -sV ... -oN Nmap/service-scan.txt`	7.50 s	Same result, saved to file
`sudo nmap -A testphp.vulnweb.com`	71.15 s	No exact OS match; 2-hop traceroute
`sudo nmap -A ... -oN Nmap/aggressive-scan.txt`	64.21 s	Same result, saved to file

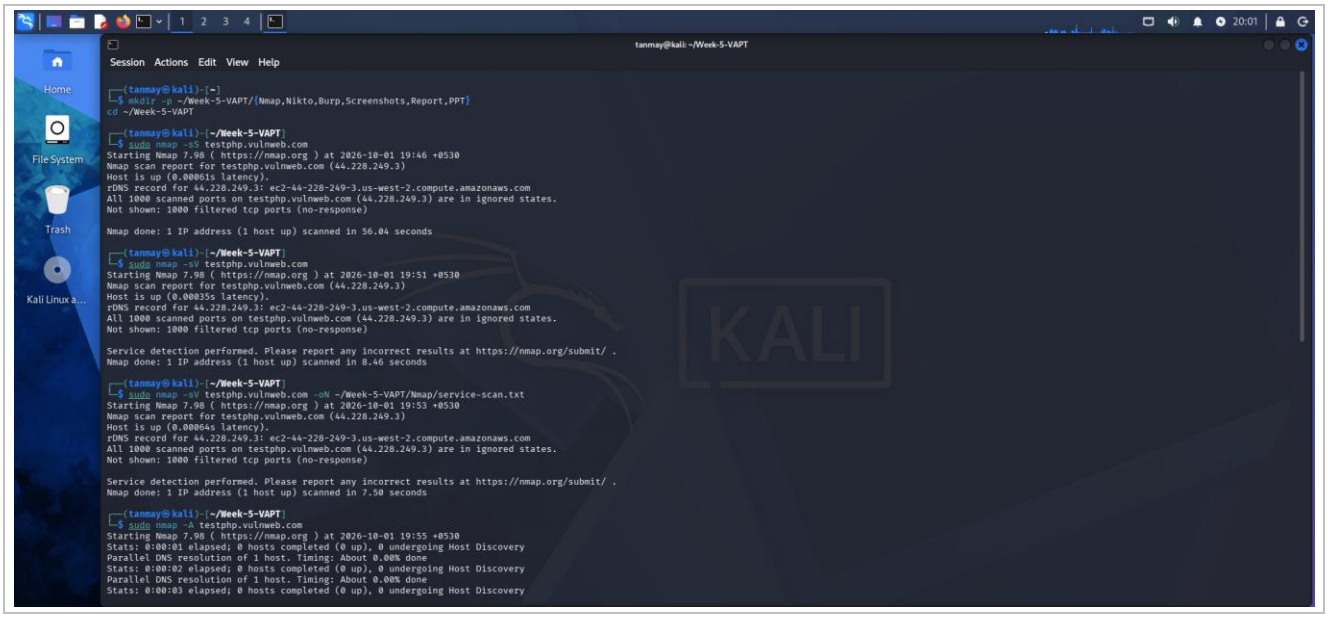


Figure 1: *Nmap -sS and -sV scans (including the saved service-scan.txt run) and the start of the -A scan against testphp.vulnweb.com.*



Figure 2: *Nmap -A results: unreliable OS guesses, 2-hop traceroute via port 80/tcp, and the repeat run saved to aggressive-scan.txt.*

3.2 Interpretation

- **Target resolved and host up.** `testphp.vulnweb.com` resolved to 44.228.249.3 (reverse DNS: an Amazon EC2 host in us-west-2) and Nmap reported the host as up.
- **No open ports observed.** All 1000 scanned TCP ports were in the *filtered* state, meaning probes received no response. A filtered port does not mean the service is vulnerable or even absent; a firewall or packet filter may be dropping the traffic. With no open port, `-sV` had no service to identify.
- **OS detection is unreliable.** Nmap warned that it could not find at least one open and one closed port, and returned low-value guesses (for example a D-Link firewall, an HP printer and Windows 2000, all at 88 to 89%). These guesses must not be reported as the real operating system.
- **Path observation.** The traceroute shows two hops, with round-trip times of only 0.17 to 0.31 ms to a cloud host that is geographically distant. Such low latency suggests that a device on the local network path (gateway, firewall or lab network) is answering or dropping probes. The scan therefore describes what is reachable from this network, not necessarily the real exposure of the target.

To confirm the remote exposure, the scan should be repeated from a different network, for example with `nmap -Pn -p 80,443 -sV testphp.vulnweb.com`, and cross-checked with a browser or `curl -I http://testphp.vulnweb.com`.

4. Local Test Environment (DVWA)

For web-layer testing, DVWA was served from the tester's own machine. Apache2 and MariaDB were started with `systemctl`, and the service state was verified before scanning.

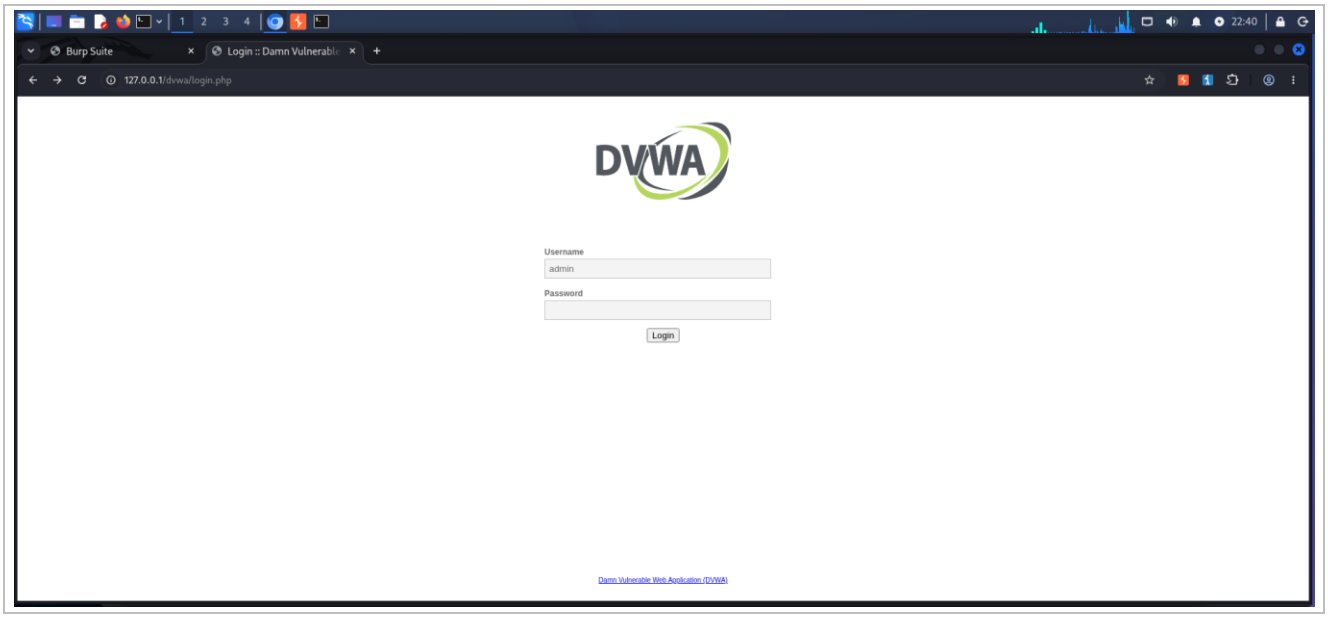


Figure 3: *DVWA login page served locally at http://127.0.0.1/dvwa/login.php (viewed through the Burp-proxied browser).*

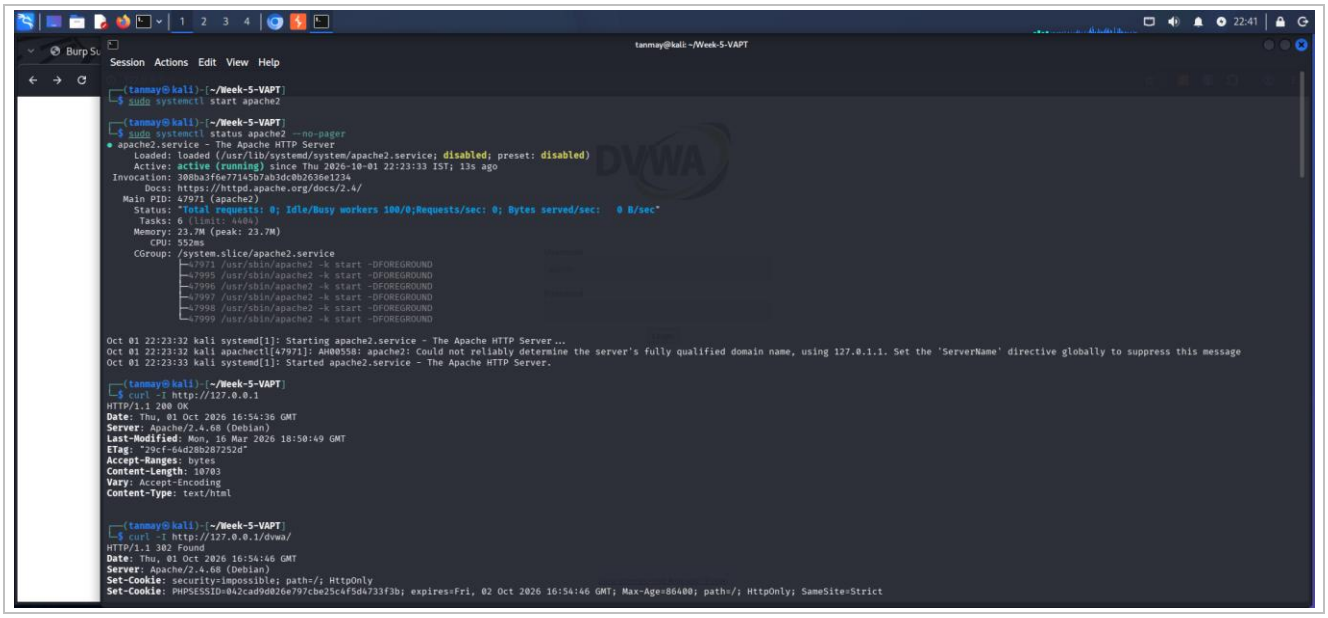


Figure 4: *Apache2 started and verified active (running); curl -I http://127.0.0.1 shows the Server header and a 200 OK response.*

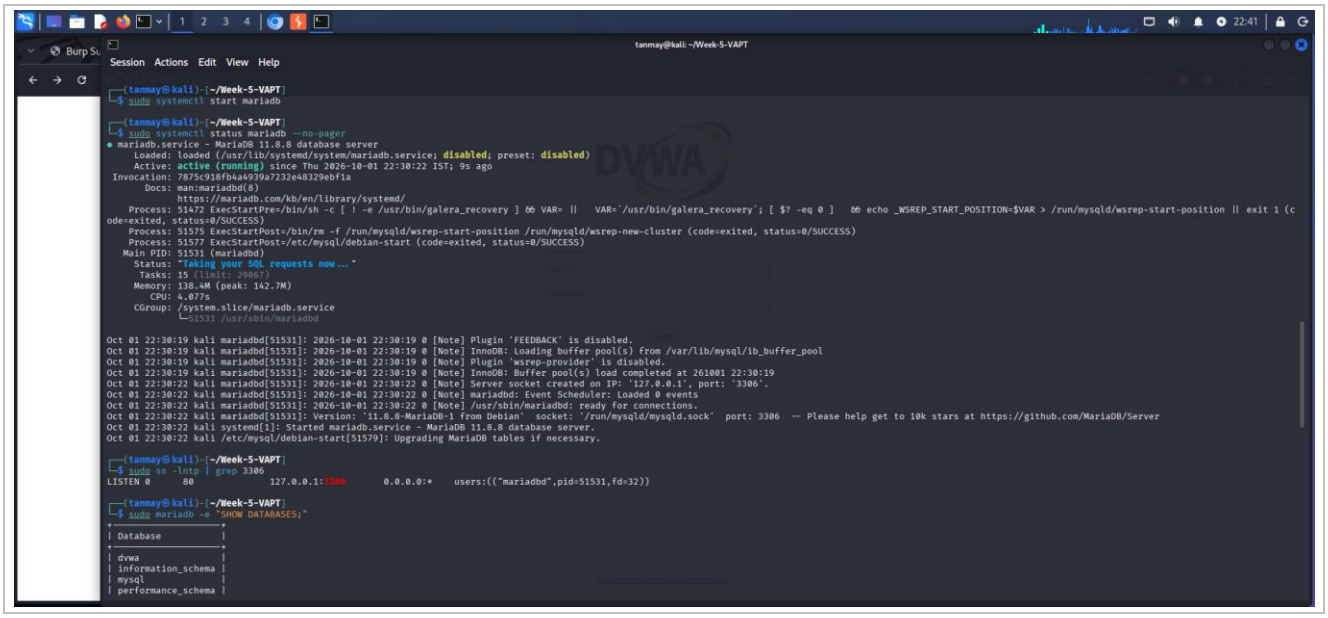


Figure 5: *MariaDB 11.8.8 running, listening only on 127.0.0.1:3306, with the dvwa database present.*

- Apache2 reported **active (running)**; its log shows the AH00558 warning that no global `ServerName` is set (a minor configuration notice).
- MariaDB listens on **127.0.0.1:3306 only**, so the database is not reachable from other hosts. This is a good control.
- The `curl -I` response exposes `Server: Apache/2.4.68 (Debian)` (see finding F-07).

5. Nikto: Web-Server Scanning

Nikto checks a web server for risky files, directories, outdated components, insecure configuration and missing headers. It was run against the DVWA path using:

```
nikto -h http://127.0.0.1/dvwa/
```

The scan was run twice. The second run saved its output with `nikto -h http://127.0.0.1/dvwa/ 2>&1 | tee ~/Week-5-VAPT/nikto-dvwa.txt`. Both runs produced identical results (8040 requests, 16 errors, 20 items), which shows the findings are repeatable.

Run	Time (IST) and duration	Result
Run 1	22:42:15 to 22:48:09 (354 s)	20 items reported
Run 2 (saved)	22:57:32 to 23:03:23 (351 s)	20 items reported; file nikto-dvwa.txt (3.3 KB)

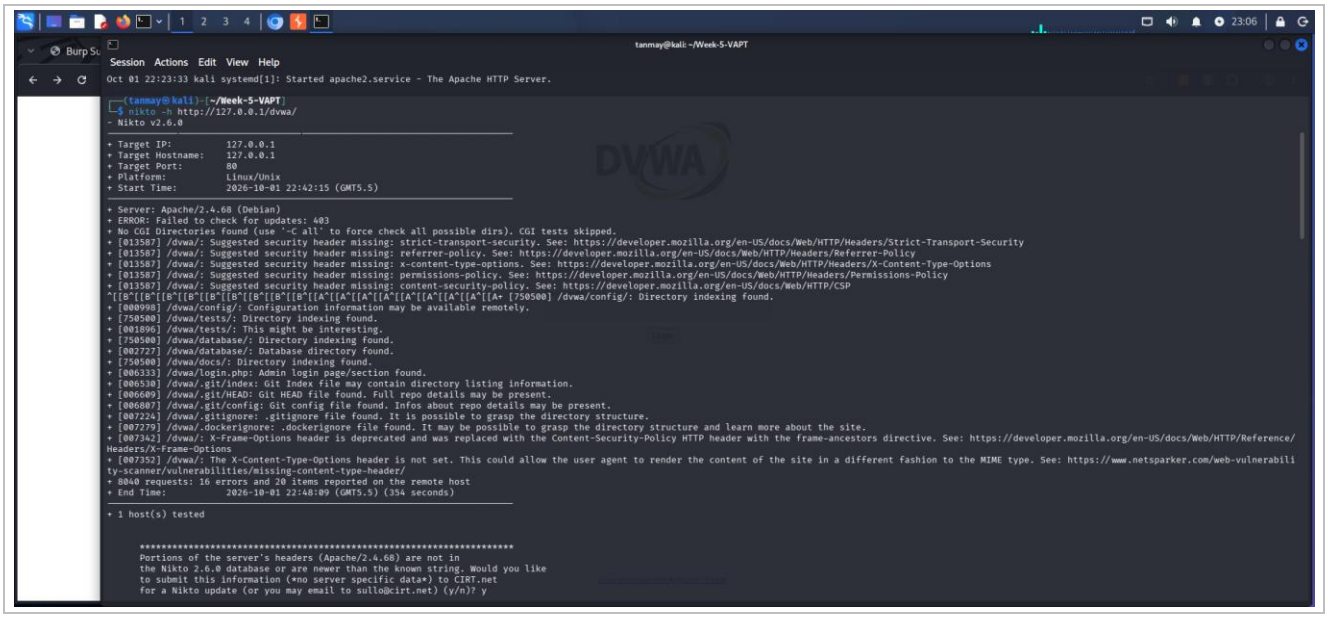


Figure 6: *Nikto v2.6.0 scan of http://127.0.0.1/dvwa/ (run 1): server banner, missing headers, directory indexing and exposed .git files.*

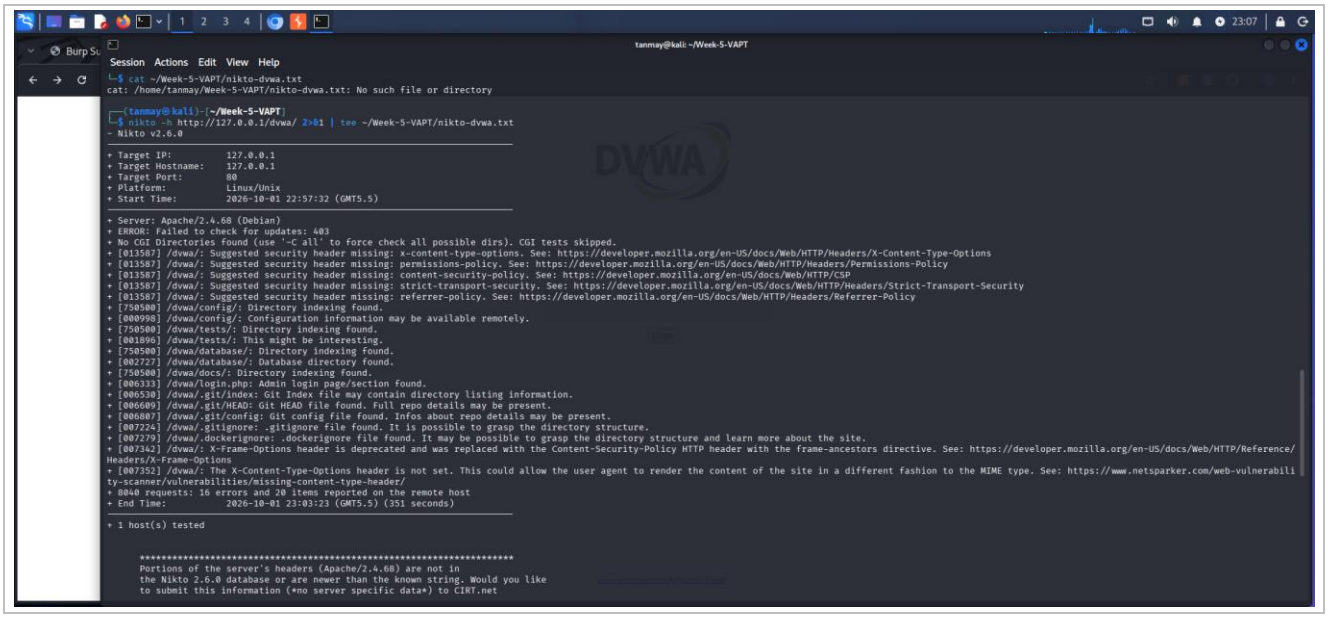


Figure 7: *Nikto re-run with output redirected to ~/Week-5-VAPT/nikto-dvwa.txt using tee (run 2).*

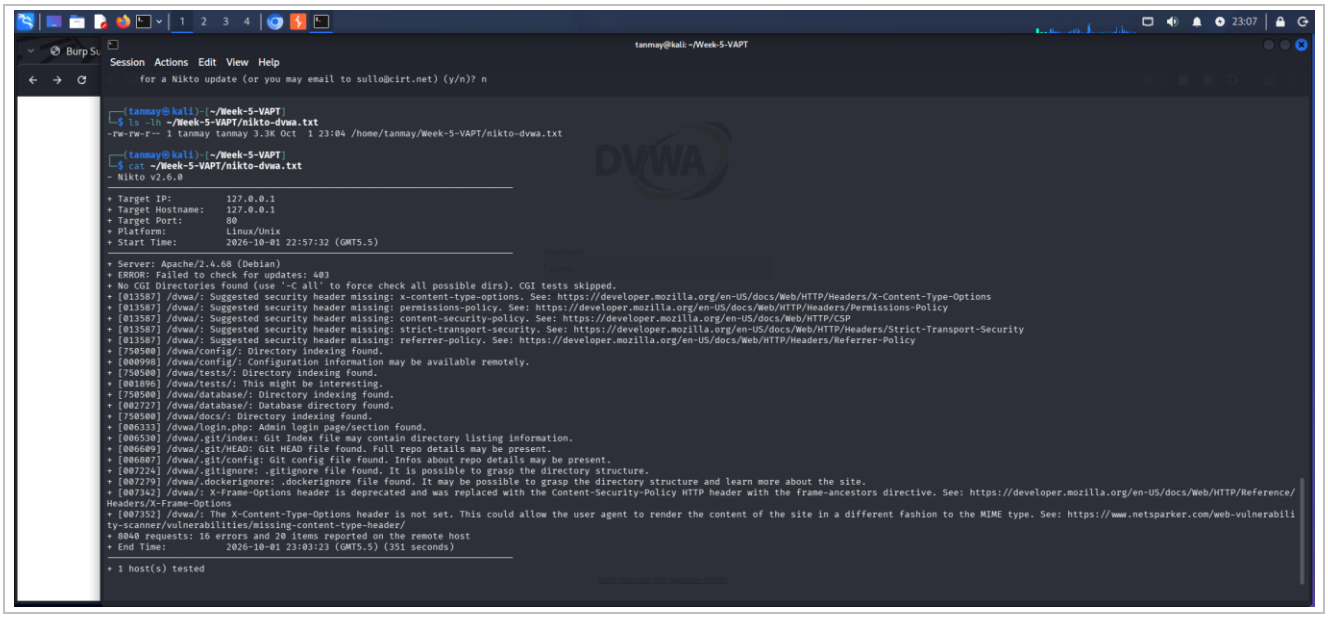


Figure 8: *Saved Nikto report displayed with cat after confirming the file with ls -lh (3.3 KB).*

5.1 Nikto results and what needs validation

Nikto result	Meaning	Validation needed
Missing headers: x-content-type-options, permissions-policy, content-security-policy, strict-transport-security, referrer-policy	The browser receives no extra hardening instructions from the server.	Confirmed manually with curl -I (Section 7).
Directory indexing: /config/, /tests/, /database/, /docs/	Folder contents may be listable by any visitor.	Open each URL in a browser to confirm the listing.
/config/: configuration information may be available; /database/: database directory found; /tests/: might be interesting	Sensitive-looking folders are reachable.	Check which files are actually readable.
/.git/index, /.git/HEAD, /.git/config, /.gitignore, /.dockerignore	Development metadata is in the web root.	Confirm the files return content (HTTP 200).
/login.php: admin login page found	Expected for the application; informational.	None.
X-Frame-Options deprecated; X-Content-Type-Options not set	Clickjacking and MIME-sniffing protection should come from CSP and nosniff.	Confirmed manually (Section 7).
Failed to check for updates: 403; no CGI directories found	Tool notes; not target findings.	None.

Nikto output is scanner evidence, not proof of exploitation. A listed file can be intentionally public, and the Apache 2.4.68 string was not in Nikto's database (the tool offered to submit it), so Nikto could not judge whether the server version is outdated.

6. Burp Suite: HTTP Traffic Analysis

Burp Suite's Proxy sits between the browser and the web application and records each request and response in **Proxy > HTTP history**. The browser was routed through the local listener `127.0.0.1:8080`, and normal DVWA pages were visited. Two requests were captured for analysis: a login form submission and the page requested after login.

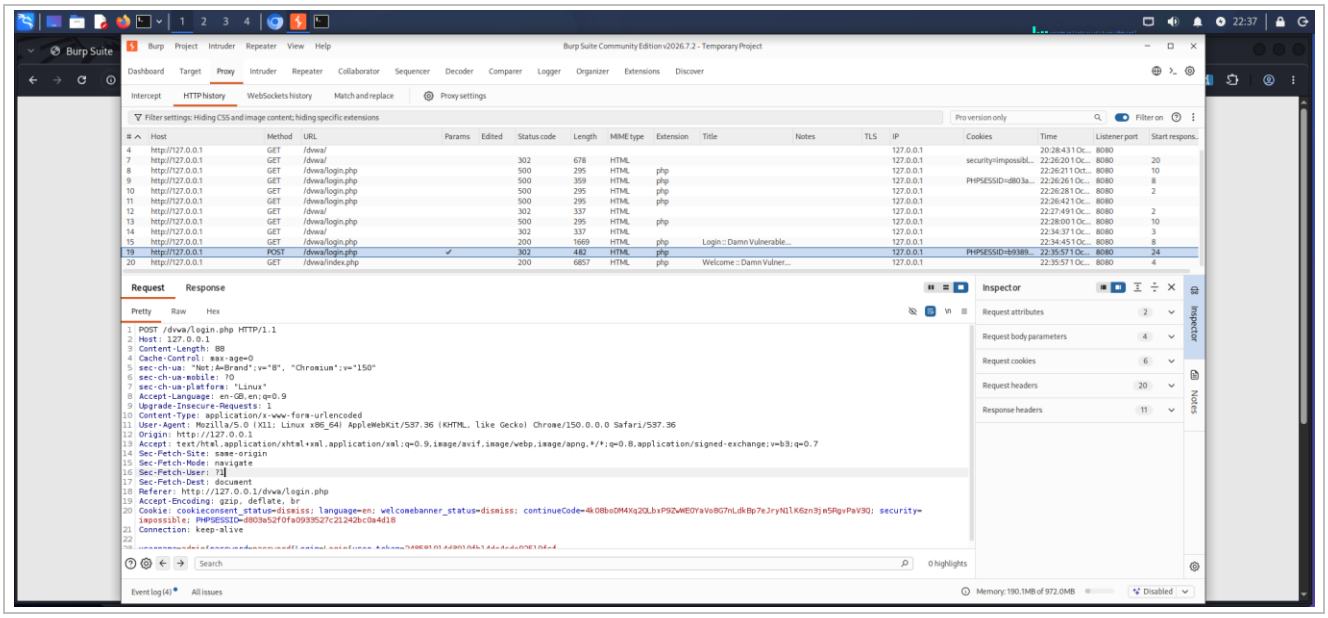


Figure 9: *Burp Suite HTTP history with request #19, POST /dvwa/login.php, selected (request tab, Pretty view).*

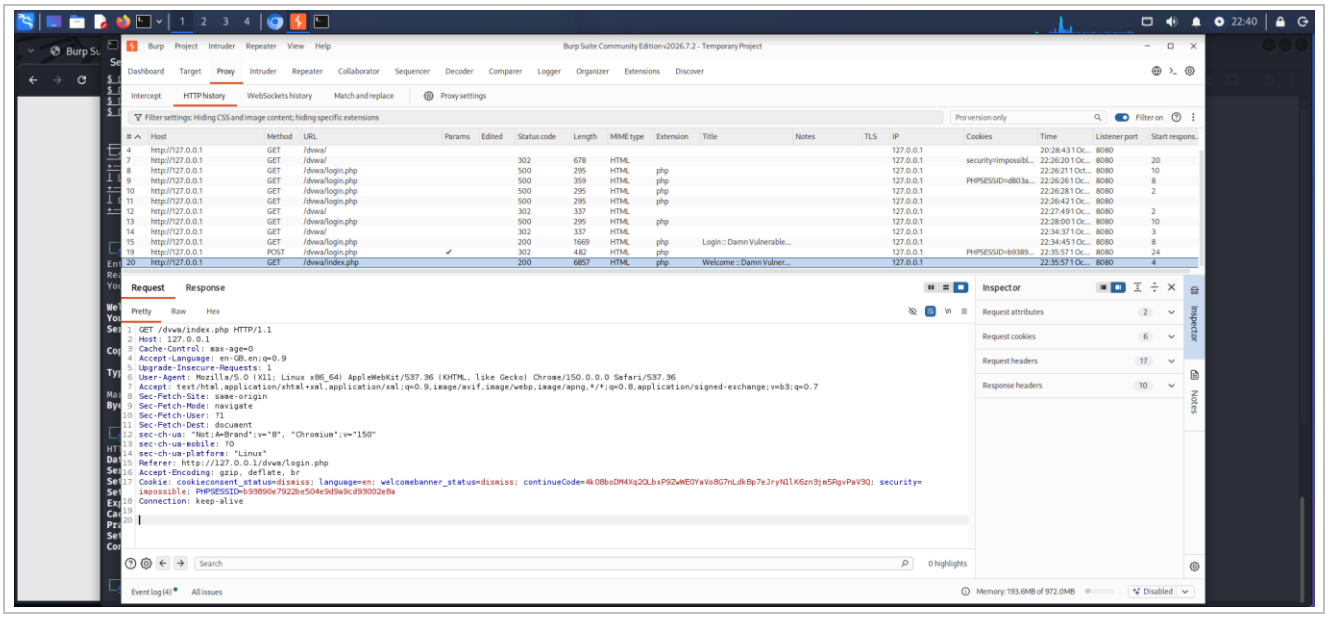


Figure 10: *Burp Suite HTTP history with request #20, GET /dvwa/index.php, selected.*

6.1 Captured requests

Attribute	Request #19	Request #20
Method and path	POST /dvwa/login.php	GET /dvwa/index.php
Host	127.0.0.1	127.0.0.1
Status / length	302 Found / 482 bytes	200 OK / 6857 bytes
MIME type / title	HTML (redirect)	HTML, "Welcome :: Damn Vulnerable..."
Content-Type	application/x-www-form-urlencoded (Content-Length 88)	Not applicable (no body)
Cookies sent	6 cookies, including PHPSESSID and security	6 cookies, including PHPSESSID and security
Header counts	20 request, 11 response headers	17 request, 10 response headers
TLS	No (plain HTTP)	No (plain HTTP)

Attribute	Request #19	Request #20
Purpose	Submits the login form. The body has 4 parameters, one being an anti-CSRF token (user_token).	Loads the home page after the redirect, now with an authenticated session.

6.2 Selected HTTP history entries

#	Method	URL	Status	Length	Note
7	GET	/dvwa/	302	678	Redirect; security cookie set
8	GET	/dvwa/login.php	500	295	Server error (database was not running yet)
9	GET	/dvwa/login.php	500	359	Server error; PHPSESSID cookie present
15	GET	/dvwa/login.php	200	1669	Login page loads after MariaDB started
19	POST	/dvwa/login.php	302	482	Login submitted, redirect to index.php
20	GET	/dvwa/index.php	200	6857	Home page after login

6.3 Interpretation

- **Methods and status codes.** GET retrieves pages; POST submits form data. 302 is a redirect (login success moves the browser to index.php), 200 is success, and 500 is a server-side failure. The 500 responses were caused by the stopped database service (see F-08).
- **Cleartext transport.** The TLS column is empty: the login POST, including the form body and session cookie, travelled over plain HTTP (F-05). The traffic stayed on the loopback interface in this lab, which limits real-world exposure.
- **Positive control.** The login form includes a per-request anti-CSRF token (`user_token`), which helps resist cross-site request forgery.
- **Cookies.** Requests carry `PHPSESSID` (session) and `security` cookies plus other application cookies; their attributes are examined in Section 7. Passwords and tokens are not reproduced in this report text.

7. Manual Header and Cookie Verification

Scanner and proxy observations were verified manually with `curl -I`, which returns only the response headers. This confirmed the missing security headers, the server banner and the cookie attributes.

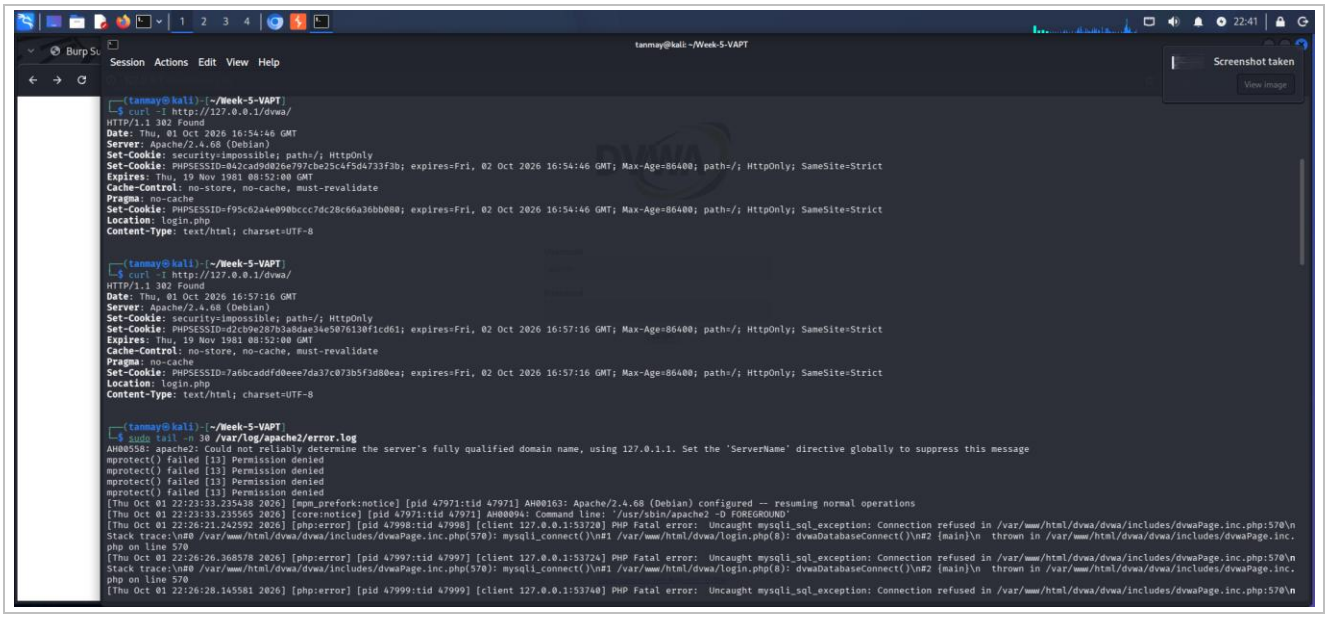


Figure 11: *curl -I http://127.0.0.1/dvwa/ (two runs) showing Set-Cookie headers and the response headers, followed by the Apache error.log tail (PHP fatal errors).*

7.1 Response headers

Across the `/`, `/dvwa/` and `/dvwa/login.php` responses, the headers present were Date, Server, Cache-Control, Pragma, Expires, Content-Type, Location (on redirects) and Set-Cookie. **None of the recommended security headers** (Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, Referrer-Policy, Permissions-Policy) were returned, which matches Nikto.

7.2 Cookie attributes

Cookie	HttpOnly	SameSite	Secure	Observation
PHPSESSID	Yes	Strict	No	Max-Age 86400; Secure flag absent (site is on HTTP).
security	Yes	Not set	No	Set with path=/ and HttpOnly only.

Two different `PHPSESSID` values were issued in a single response; the browser keeps the later one. This is worth reviewing in the application's session handling, but it is not a vulnerability by itself.

7.3 Apache error log

The Apache `error.log` recorded PHP fatal errors (`mysqli_sql_exception: Connection refused`) with full stack traces and file paths such as `/var/www/html/dvwa/dvwa/includes/dvwaPage.inc.php:570`. The cause was the stopped MariaDB service; starting it resolved the 500 errors seen in Burp. Log content is not shown to site visitors, but it does reveal internal paths if logs or error display are exposed.

8. Application and Database Configuration Review

As the owner of the lab machine, the tester reviewed the DVWA configuration file locally (with `sudo`) to understand what an attacker would gain if that file were exposed.

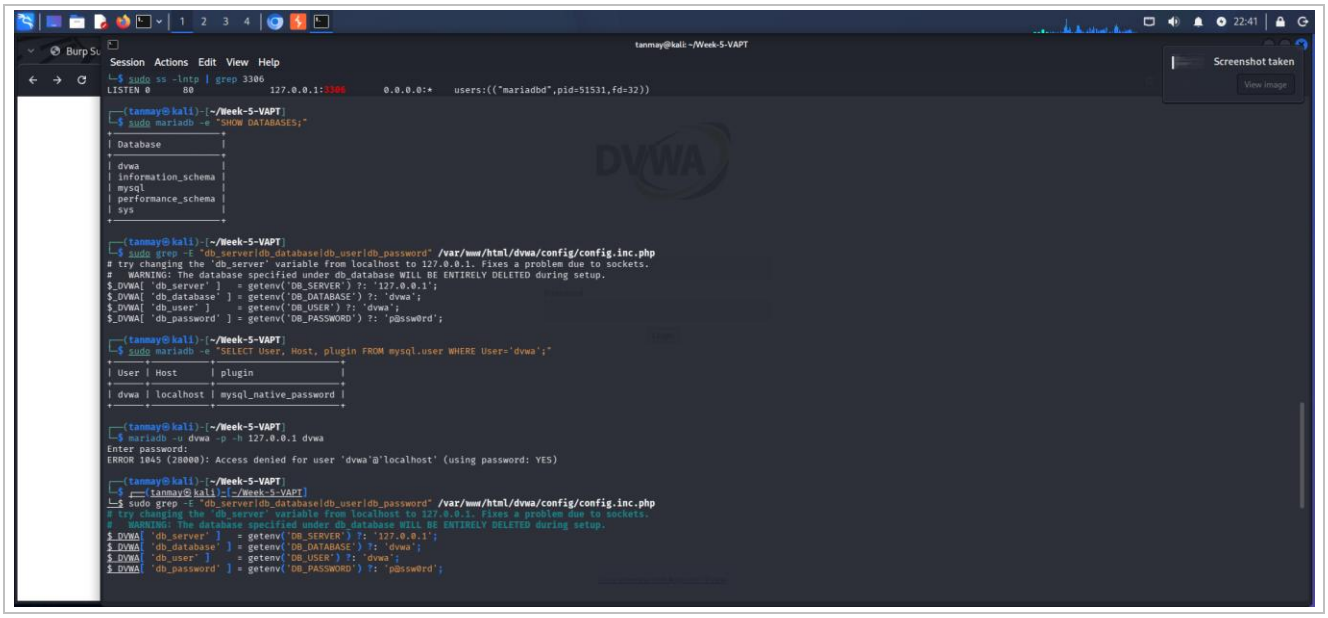


Figure 12: *config.inc.php database settings, the dvwa account lookup in mysql.user, and a first login attempt rejected with ERROR 1045.*

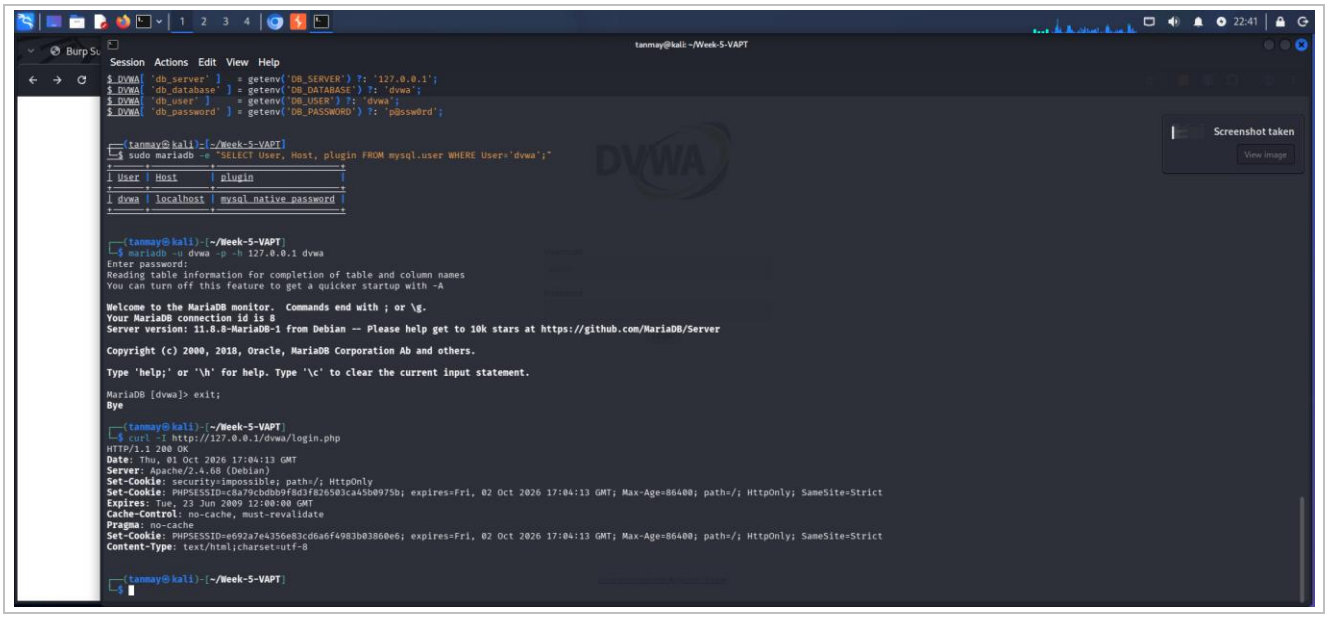


Figure 13: *Successful MariaDB login with the dvwa account, followed by curl -I http://127.0.0.1/dvwa/login.php.*

- `config.inc.php` defines the database server (127.0.0.1), database (dvwa), user (dvwa) and a **default password** that is publicly documented for DVWA. The file supports environment-variable overrides (`getenv`), but the fallback default was in use. The password value is intentionally not repeated in this text.
- The `dvwa@localhost` account uses `mysql_native_password`. A later login attempt succeeded, which is consistent with the configured credentials being valid.
- This review needed local privileged access. It shows the impact if the file leaked; it does not show that the file is remotely readable.

9. Findings and Recommendations

Severity follows the scale defined for this practical (Informational, Low, Medium, High, Critical). Ratings reflect evidence, exploitability, affected asset and impact. No High or Critical findings were supported by the evidence collected.

9.1 Findings summary

ID	Finding	Tool	Severity	Validation
F-01	Directory indexing on /config/, /tests/, /database/, /docs/	Nikto	Medium	Scanner-reported, repeated
F-02	Exposed .git and ignore files	Nikto	Medium	Scanner-reported, repeated
F-03	Default database credentials in config	Manual review	Medium	Confirmed locally
F-04	Missing HTTP security headers	Nikto, curl	Low	Confirmed
F-05	Login sent over unencrypted HTTP	Burp Suite	Low	Confirmed
F-06	Cookie attribute gaps (Secure, SameSite)	curl	Low	Confirmed
F-07	Server version disclosure	Nikto, curl	Low	Confirmed
F-08	Server errors and verbose error logs	Burp, log	Informational	Confirmed
F-09	No open TCP ports observed (all filtered)	Nmap	Informational	Repeated 5 times

9.2 Detailed findings

F-01 | Directory indexing enabled	Severity: Medium
Affected / Tool	/dvwa/config/, /dvwa/tests/, /dvwa/database/, /dvwa/docs/ (Nikto ID 750500)
Observation and evidence	Nikto reported "Directory indexing found" on four directories, plus "Configuration information may be available remotely" for /config/ and "Database directory found" for /database/. Both runs gave the same result (Figures 6 to 8).
Security impact	Visitors may list and download files that should not be public. Folder names alone help an attacker find configuration, database scripts and test files, and any readable file there could expose credentials or application internals.
Recommendation	Disable automatic listing (`Options -Indexes`). Remove or deny web access to config, tests, database and docs folders in any production deployment.
Validation status	Scanner-reported and reproduced in two runs. Manual browser confirmation of each URL is recommended before final rating.

F-02 | Version-control and build metadata exposed	Severity: Medium
Affected / Tool	/dvwa/.git/index, /.git/HEAD, /.git/config, /.gitignore, /.dockerignore (Nikto IDs 006530 to 007279)
Observation and evidence	Nikto found Git repository files and ignore files in the web root (Figures 6 to 8).
Security impact	An exposed .git folder can allow the source repository, history and any committed secrets to be reconstructed. Ignore files reveal the directory structure.
Recommendation	Remove .git and other development files from the deployed web root, block dot-files in the server configuration, and deploy from build artifacts instead of a repository checkout.
Validation status	Scanner-reported and reproduced. The files were not downloaded, so retrieval should be confirmed (HTTP 200) before final rating.

F-03 | Default database credentials in application configuration	Severity: Medium
Affected / Tool	/var/www/html/dvwa/config/config.inc.php; MariaDB account dvwa@localhost (manual review)
Observation and evidence	The file falls back to a publicly documented default password. A MariaDB login with the dvwa account succeeded (Figures 12 and 13). MariaDB listens only on 127.0.0.1:3306 (Figure 5).

Security impact	Anyone who can read the file (see F-01) or reach the database could use known credentials. The localhost-only listener is a mitigating control.
Recommendation	Set a unique strong password supplied through environment variables or a secrets store, grant the account only the privileges it needs, and keep config folders out of the web root.
Validation status	Confirmed locally with privileged access. Remote readability of the file was not demonstrated.

F-04 | Missing HTTP security headers	**Severity: Low**
Affected / Tool	http://127.0.0.1/ and /dvwa/ (Nikto ID 013587, 007352; curl -I)
Observation and evidence	Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, Referrer-Policy and Permissions-Policy are absent. Nikto also notes that X-Frame-Options is deprecated in favour of CSP frame-ancestors (Figures 4, 6 and 11).
Security impact	Weaker browser-side protection against cross-site scripting, clickjacking, MIME sniffing and referrer leakage. This is a hardening gap, not a directly exploitable flaw.
Recommendation	Add headers with mod_headers after testing: `X-Content-Type-Options: nosniff`, a Referrer-Policy, a Content-Security-Policy including `frame-ancestors`, and HSTS once HTTPS is enabled.
Validation status	Confirmed by Nikto and manual curl -I. HSTS applies only after HTTPS is in place.

F-05 | Login credentials sent over unencrypted HTTP	**Severity: Low**
Affected / Tool	POST /dvwa/login.php (Burp request #19)
Observation and evidence	The login form and session cookie are sent over http:// with no TLS (Figure 9).
Security impact	On a shared or untrusted network, credentials and session cookies could be captured. Here the traffic stays on the loopback interface, so risk is low; on a real network it would be High.
Recommendation	Enable HTTPS with a valid certificate, redirect HTTP to HTTPS, enable HSTS and mark cookies Secure.
Validation status	Confirmed in Burp HTTP history.

F-06 | Cookie attribute gaps	**Severity: Low**
Affected / Tool	PHPSESSID and security cookies (curl -I /dvwa/)
Observation and evidence	Both cookies carry HttpOnly. PHPSESSID has SameSite=Strict. Neither has the Secure flag, and the security cookie has no SameSite attribute (Figure 11).
Security impact	Without Secure, cookies may be sent over plain HTTP; without SameSite, some cross-site request risks remain. No session compromise was attempted or demonstrated.
Recommendation	Set Secure and SameSite on all cookies (for example `session.cookie_secure`, `session.cookie_samesite` in php.ini) once HTTPS is enabled.
Validation status	Confirmed from Set-Cookie headers.

F-07 | Server version disclosure	**Severity: Low**
Affected / Tool	Server header on all responses (Nikto, curl -I)
Observation and evidence	Responses contain `Server: Apache/2.4.68 (Debian)` (Figures 4 and 11).
Security impact	Reveals the web-server product, version and operating system family, helping an attacker choose targeted attacks. It does not prove the version is vulnerable.

Recommendation	Set `ServerTokens Prod` and `ServerSignature Off`, keep Apache patched, and compare the version against vendor security advisories.
Validation status	Confirmed. Nikto did not recognize this version string, so outdated status was not assessed.

F-08 | Server errors and verbose error logging	**Severity: Informational**
Affected / Tool	/dvwa/login.php (Burp #8 to #13, Apache error.log)
Observation and evidence	HTTP 500 responses (295 to 359 bytes) were returned while MariaDB was stopped; the error log contains full PHP stack traces with file paths (Figures 9 and 11).
Security impact	Internal paths and code structure can leak if error display or log files are exposed. Whether error details reached users was not verified.
Recommendation	Set `display_errors=Off` and `log_errors=On`, use custom error pages, restrict log access and monitor database availability.
Validation status	Confirmed in Burp history and logs.

F-09 | No open TCP ports observed on testphp.vulnweb.com	**Severity: Informational**
Affected / Tool	testphp.vulnweb.com, 44.228.249.3 (Nmap -sS, -sV, -A)
Observation and evidence	All 1000 scanned TCP ports were filtered; OS guesses were unreliable; round-trip times of 0.17 to 0.31 ms suggest filtering on the local path (Figures 1 and 2).
Security impact	Nothing could be concluded about the remote services. The result may reflect the lab network rather than the target.
Recommendation	Repeat from a different network with `-Pn -p 80,443 -sV` and verify with a browser or curl before drawing conclusions.
Validation status	Result was consistent across five scans; real exposure not verified.

9.3 Remediation priorities

- **First:** remove or block /config/, /tests/, /database/, /docs/ and .git (F-01, F-02), and replace default database credentials (F-03).
- **Next:** enable HTTPS, then add HSTS, Secure and SameSite cookie settings (F-04, F-05, F-06).
- **Hardening:** reduce banner information and error output (F-07, F-08).

10. Conclusion, Limitations and References

10.1 Conclusion

This practical applied the basic VAPT workflow using Nmap, Nikto and Burp Suite. Nmap showed that the remote target did not respond on any of the 1000 scanned ports from this network. Nikto and Burp Suite, used against a locally hosted DVWA instance, produced nine documented findings, led by directory indexing, exposed repository metadata and default database credentials. Manual checks with curl confirmed the missing headers, server banner and cookie attributes.

The main lesson is that running a tool is the start of the work: every result has to be interpreted, validated, rated and tied to a realistic impact. The practical also reinforced that testing must stay within permission and scope.

10.2 Limitations

- Automated scanners produce false positives and miss issues; this is not a complete security audit.

- The Nmap result may be influenced by network filtering on the lab path and was not cross-checked from another network.
- Nikto and Burp Suite were applied to a local, intentionally vulnerable application, so the findings illustrate method and not the security of the Acunetix site.
- Several scanner-reported items (F-01, F-02) still need manual retrieval to be fully confirmed.

10.3 References

- Nmap Reference Guide: https://nmap.org/book/man.html
- Nikto web server scanner: https://github.com/sullo/nikto
- PortSwigger, Burp Suite Proxy documentation: https://portswigger.net/burp/documentation/desktop/tools/proxy
- OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/
- MDN Web Docs, Set-Cookie header: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie
- Damn Vulnerable Web Application (DVWA): https://github.com/digininja/DVWA
- Acunetix Vulnweb test site: http://testphp.vulnweb.com

Appendix A: Evidence index

Figure	Content	Source file
1	Nmap -sS, -sV, start of -A	Screenshot_2026-10-01_20_01_01.png
2	Nmap -A results and saved run	Screenshot_2026-10-01_20_01_19.png
3	DVWA login page	Screenshot_2026-10-01_22_40_27.png
4	Apache status and curl -I	Screenshot_2026-10-01_22_41_17.png
5	MariaDB status and databases	Screenshot_2026-10-01_22_41_36.png
6	Nikto run 1	Screenshot_2026-10-01_23_06_58.png
7	Nikto run 2 with tee	Screenshot_2026-10-01_23_07_15.png
8	Saved Nikto report	Screenshot_2026-10-01_23_07_32.png
9	Burp request #19 (POST login)	Screenshot_2026-10-01_22_37_34.png
10	Burp request #20 (GET index.php)	Screenshot_2026-10-01_22_40_10.png
11	curl cookies and Apache error log	Screenshot_2026-10-01_22_41_22.png
12	config.inc.php and DB user check	Screenshot_2026-10-01_22_41_44.png
13	MariaDB login and login.php headers	Screenshot_2026-10-01_22_41_47.png
Text	Nmap, Nikto output files	service-scan.txt, aggressive-scan.txt, nikto-dvwa.txt